ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ «ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Московский институт электроники и математики им. А.Н. Тихонова

ИНТЕЛЛЕКТУАЛЬНАЯ СИСТЕМА АВТОМАТИЗАЦИИ МОНИТОРИНГА И РЕАГИРОВАНИЯ НА ИНЦИДЕНТЫ ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

Выпускная квалификационная работа — магистерская диссертация

по направлению 10.04.01 «Информационная безопасность»

студента образовательной программы магистратуры «Кибербезопасность»

Москва 2026

Аннотация

Настоящая выпускная квалификационная работа посвящена разработке интеллектуальной системы автоматизации мониторинга и реагирования на инциденты информационной безопасности. Система интегрирует SIEM-платформу Wazuh, подсистему контекстного обогащения на основе osquery, механизм многоуровневой корреляции событий и модуль интеллектуального анализа, использующий локально развёрнутую большую языковую модель. Ключевой принцип разработки состоит в том, что языковая модель выступает интеллектуальным помощником аналитика, не принимая автономных решений и не выполняя автоматических действий реагирования; итоговое решение всегда остаётся за человеком.

В работе проведён анализ предметной области, обоснован выбор технологий и архитектурных решений, спроектирована и реализована многокомпонентная система, функционирующая в изолированном облачном тестовом стенде. Разработаны алгоритмы нормализации событий, адаптивного выбора обогащающих запросов, темпоральной и контекстной корреляции, обнаружения цепочек тактик MITRE ATT&CK, детерминированной оценки критичности и формирования рекомендаций по реагированию. Реализован трёхрежимный конвейер анализа: детерминированный базовый, с использованием языковой модели и с полным контекстным обогащением. Проведена экспериментальная оценка эффективности системы на корпусе из двадцати размеченных инцидентов, включающая сравнение режимов работы по метрикам точности классификации, полноты выявления индикаторов и корректности рекомендаций реагирования.

Работа содержит __ страниц, __ рисунков, __ таблиц, __ источников, __ приложений.

Abstract

This Master's thesis presents the design and implementation of an intelligent system for automating information security monitoring and incident response. The system integrates the Wazuh SIEM platform, an osquery-based host context enrichment subsystem, a multi-level event correlation engine, and an intelligent analysis module powered by a locally deployed large language model. The core design principle ensures that the language model serves as an intelligent assistant to the analyst without making autonomous decisions or triggering automated response actions; the final judgment always remains with the human operator.

The work includes a domain analysis, a justified selection of technologies and architectural decisions, as well as the design and implementation of a multi-component system operating within an isolated cloud-based test environment. Algorithms for event normalization, adaptive enrichment query selection, temporal and contextual correlation, MITRE ATT&CK tactic chain detection, deterministic criticality scoring, and response recommendation generation have been developed. A three-mode analysis pipeline has been implemented: deterministic baseline, LLM-assisted, and fully context-enriched LLM analysis. An experimental evaluation of system effectiveness has been conducted on a corpus of twenty labeled incidents, comparing operational modes across classification accuracy, indicator coverage, and response recommendation correctness metrics.

Содержание

1 Введение

2 Анализ предметной области и обзор существующих решений

- 2.1 Проблематика мониторинга инцидентов информационной безопасности
- 2.2 Обзор систем управления событиями информационной безопасности
- 2.3 Применение больших языковых моделей в задачах информационной безопасности
- 2.4 Средства контекстного обогащения событий на уровне хоста
- 2.5 Корреляция событий информационной безопасности
- 2.6 Анализ существующих решений автоматизации реагирования
- 2.7 Постановка задачи и обоснование необходимости разработки
- 2.8 Выводы по главе

3 Проектирование интеллектуальной системы

- 3.1 Формулировка требований к системе
- 3.2 Общая архитектура системы
- 3.3 Проектирование подсистемы контекстного обогащения
- 3.4 Проектирование механизма корреляции событий
- 3.5 Проектирование модуля интеллектуального анализа и оценки инцидентов
- 3.6 Алгоритмическое обеспечение системы
- 3.7 Выводы по главе

4 Реализация и экспериментальная оценка

- 4.1 Технологический стек и обоснование выбора инструментов
- 4.2 Развёртывание тестового стенда
- 4.3 Реализация серверной части и уровня данных
- 4.4 Реализация подсистемы контекстного обогащения
- 4.5 Реализация механизма корреляции
- 4.6 Реализация модуля интеллектуального анализа
- 4.7 Пользовательский интерфейс
- 4.8 Тестирование
- 4.9 Экспериментальная оценка эффективности
- 4.10 Направления дальнейшего развития
- 4.11 Выводы по главе

5 Заключение

Список использованных источников

ПРИЛОЖЕНИЕ А. Структура программного проекта

ПРИЛОЖЕНИЕ Б. Диаграмма компонентов системы

ПРИЛОЖЕНИЕ В. Формат структурированного результата анализа

ПРИЛОЖЕНИЕ Г. Конфигурация тестового стенда

---

1 Введение

Современная информационная инфраструктура организаций находится под непрерывным воздействием всё более изощрённых угроз информационной безопасности. Центры мониторинга и реагирования на инциденты (SOC) играют ключевую роль в обеспечении защиты, однако их эффективность существенно ограничена высоким объёмом генерируемых оповещений. По данным исследований Ponemon Institute и IBM, среднее время обнаружения и локализации инцидента информационной безопасности составляет более двухсот дней, при этом значительная часть рабочего времени аналитиков расходуется на рутинный первичный разбор событий, большинство из которых оказываются ложноположительными либо не требуют немедленной реакции [1].

Системы управления событиями информационной безопасности (SIEM) обеспечивают централизованный сбор и хранение журналов событий, однако нагрузка на аналитика остаётся значительной: каждое оповещение требует ручного сбора контекста, сопоставления с известными шаблонами атак, оценки критичности и формирования рекомендации по реагированию. Автоматизация этих процессов является актуальной научно-технической задачей, решение которой способно существенно ускорить начальный этап разбора инцидентов и снизить вероятность пропуска критически важных событий.

Появление больших языковых моделей (БЯМ) открывает новые возможности для интеллектуальной обработки текстовых данных, включая журналы событий и оповещения SIEM-систем. Тем не менее применение облачных языковых моделей для анализа данных информационной безопасности сопряжено с рисками утечки конфиденциальной телеметрии за периметр организации. Локальное

развёртывание моделей на оборудовании аналитика позволяет устранить этот риск, сохраняя доступ к возможностям интеллектуального анализа текста.

Одновременно с развитием инструментов анализа растёт понимание важности контекстного обогащения событий. Изолированное рассмотрение единичного оповещения без учёта состояния хоста, активных процессов, сетевых соединений и связанных событий затрудняет корректную оценку инцидента. Интеграция средств сбора системной телеметрии с механизмами корреляции позволяет сформировать более полную картину инцидента и повысить качество итогового анализа.

Целью настоящей работы является разработка и реализация интеллектуальной системы автоматизации мониторинга и реагирования на инциденты информационной безопасности, объединяющей подсистему контекстного обогащения и корреляции данных с программным модулем интеллектуального анализа событий на основе локально развёрнутой большой языковой модели.

Для достижения поставленной цели сформулированы следующие **задачи**:

1. Провести анализ предметной области: исследовать существующие подходы к мониторингу и реагированию на инциденты информационной безопасности, проанализировать возможности применения больших языковых моделей в задачах первичного разбора оповещений SIEM-систем, оценить средства контекстного обогащения и корреляции событий.

2. Сформулировать требования к интеллектуальной системе автоматизации мониторинга и реагирования, определить архитектуру системы и состав её компонентов.

3. Разработать подсистему контекстного обогащения событий, обеспечивающую сбор системной информации с хостов и связывание полученных данных с оповещениями SIEM-системы.

4. Разработать механизм многоуровневой корреляции событий, включающий темпоральный, контекстный и тактический анализ по матрице MITRE ATT&CK.

5. Разработать программный модуль интеллектуального анализа, обеспечивающий оценку критичности инцидента и формирование рекомендаций по выбору способа реагирования с использованием локально развёрнутой большой языковой модели.

6. Реализовать прототип системы в виде действующего программного комплекса и провести экспериментальную оценку его эффективности на подготовленном тестовом стенде.

Объектом исследования является процесс мониторинга и реагирования на инциденты информационной безопасности в условиях высокого объёма оповещений.

Предметом исследования являются методы и средства интеллектуальной автоматизации первичного разбора оповещений информационной безопасности с использованием контекстного обогащения, корреляции событий и языковых моделей.

Научная новизна работы определяется следующими положениями:

- предложен подход к интеграции локально развёрнутой большой языковой модели в конвейер анализа событий SIEM-системы с механизмом многоуровневой корреляции и контекстного обогащения;

- разработан трёхрежимный конвейер анализа инцидентов, позволяющий сравнить эффективность детерминированного, модельного и обогащённого методов оценки;
- предложена методика экспериментальной оценки качества интеллектуального анализа оповещений на основе размеченного корпуса инцидентов.

Практическая значимость работы заключается в создании действующего программного прототипа, который может быть использован в учебных и исследовательских целях для изучения возможностей применения языковых моделей в центрах мониторинга информационной безопасности, а также в качестве основы для построения промышленных решений автоматизации первичного разбора инцидентов.

Работа состоит из введения, трёх основных глав, заключения, списка использованных источников и четырёх приложений. В первой главе проведён аналитический обзор предметной области. Во второй главе описаны архитектурные и алгоритмические решения, положенные в основу системы. В третьей главе рассмотрена реализация, тестирование и экспериментальная оценка разработанного прототипа.

---

2 Анализ предметной области и обзор существующих решений

2.1 Проблематика мониторинга инцидентов информационной безопасности

Центры мониторинга и реагирования на инциденты информационной безопасности (Security Operations Center, SOC) представляют собой организационные и технические структуры, ответственные за непрерывный контроль состояния защищённости информационных систем, выявление инцидентов и координацию действий по их устранению [2]. Деятельность SOC включает сбор и анализ журналов событий, выявление аномалий, классификацию инцидентов по степени критичности и формирование рекомендаций по реагированию.

Ключевой инструмент SOC — системы управления событиями информационной безопасности (Security Information and Event Management, SIEM), обеспечивающие централизованный приём, нормализацию и хранение данных от множества источников: межсетевых экранов, средств обнаружения вторжений, антивирусных решений, журналов операционных систем и прикладных сервисов [3]. SIEM-системы осуществляют корреляцию событий по заданным правилам и генерируют оповещения (алерты) при обнаружении потенциально значимых ситуаций.

Несмотря на высокую степень автоматизации сбора данных, процесс интерпретации оповещений остаётся преимущественно ручным. Аналитик SOC при получении оповещения должен выполнить последовательность действий: собрать дополнительный контекст об источнике события, сопоставить его с известными шаблонами атак и тактиками злоумышленников, оценить степень критичности и сформулировать рекомендацию по дальнейшим действиям. Эта последовательность повторяется для каждого поступающего оповещения, и в условиях крупных инфраструктур количество событий может достигать десятков тысяч в сутки [4].

Проблема усугубляется высокой долей ложноположительных срабатываний. Исследования показывают, что до восьмидесяти процентов оповещений SIEM-систем при детальном рассмотрении не представляют реальной угрозы [5]. Тем не менее каждое из них требует анализа, что создаёт эффект «усталости от оповещений» (alert fatigue) — состояние, при котором аналитик начинает пропускать или недостаточно тщательно изучать поступающие события, что повышает риск пропуска подлинных инцидентов.

Дефицит квалифицированных специалистов в области информационной безопасности — ещё один фактор, определяющий актуальность автоматизации. По оценкам (ISC)², глобальная нехватка специалистов в области кибербезопасности составляет несколько миллионов человек [6]. В таких условиях инструменты, способные разгрузить аналитика от рутинных операций и ускорить первичный разбор, приобретают особое значение.

Таким образом, задача автоматизации первичного разбора оповещений информационной безопасности включает несколько аспектов:

- сокращение времени от поступления оповещения до формирования первичной оценки;
- обогащение события контекстной информацией о состоянии затронутого хоста;
- корреляция с другими событиями для выявления скоординированных действий;
- интеллектуальная оценка критичности и формирование рекомендаций по реагированию;
- сохранение принципа «человек принимает решение» — автоматизация должна носить рекомендательный характер.

2.2 Обзор систем управления событиями информационной безопасности

Системы SIEM являются центральным элементом инфраструктуры мониторинга. На рынке представлен широкий спектр решений, различающихся по функциональности, модели поставки и стоимости.

[ЗАГЛУШКА ДЛЯ ДОПОЛНЕНИЯ: здесь необходимо добавить развёрнутое сравнительное исследование SIEM-систем. Рекомендуется рассмотреть следующие решения:

- **Коммерческие:** Splunk Enterprise Security, IBM QRadar, Microsoft Sentinel, ArcSight (Micro Focus), LogRhythm;
- **С открытым исходным кодом:** Wazuh, OSSIM (AlienVault), Security Onion, Elastic SIEM;
- **Отечественные:** MaxPatrol SIEM (Positive Technologies), KUMA (Kaspersky), RuSIEM.

Критерии сравнения должны включать: архитектуру (монолитная, распределённая, облачная), поддерживаемые источники данных, возможности корреляции и правил обнаружения, интеграционные интерфейсы (API), модель лицензирования, наличие встроенных средств автоматизации реагирования (SOAR), документированность и активность сообщества.

Результаты сравнения следует оформить в виде сводной таблицы с балльными оценками по каждому критерию. Обоснование выбора Wazuh в качестве SIEM-платформы для настоящей работы должно вытекать из этого анализа.]

Для целей настоящей работы в качестве SIEM-платформы выбрана система Wazuh — решение с открытым исходным кодом, обеспечивающее комплексный мониторинг информационной

безопасности, включая обнаружение вторжений на уровне хоста (HIDS), контроль целостности файлов, аудит конфигураций и интеграцию с матрицей MITRE ATT&CK [7].

Архитектура Wazuh включает три основных компонента:

- **Wazuh Manager** — центральный сервер, осуществляющий приём и обработку данных от агентов, применение правил обнаружения, генерацию оповещений и предоставление управляющего API;
- **Wazuh Indexer** — хранилище событий на основе OpenSearch, обеспечивающее индексацию, полнотекстовый поиск и аналитические запросы по накопленным данным;
- **Wazuh Dashboard** — веб-интерфейс визуализации, предоставляющий оператору обзор текущего состояния защищённости и инструменты ручного анализа.

Wazuh поддерживает таксономию MITRE ATT&CK: правила обнаружения аннотированы идентификаторами тактик и техник, что позволяет автоматически связывать оповещения с элементами матрицы. Уровни правил (от 0 до 15) определяют начальную критичность события и могут быть использованы для автоматической классификации.

Выбор Wazuh обусловлен следующими факторами: открытый исходный код и отсутствие лицензионных ограничений, активное сообщество и регулярные обновления, наличие RESTful API для программного взаимодействия, встроенная поддержка MITRE ATT&CK, возможность развёртывания в контейнерной среде и совместимость с OpenSearch для гибких аналитических запросов.

2.3 Применение больших языковых моделей в задачах информационной безопасности

Большие языковые модели (Large Language Models, LLM) — класс моделей глубокого обучения, основанных на архитектуре Transformer, обученных на массивных текстовых корпусах и способных генерировать связный текст, выполнять классификацию, суммаризацию и ответы на вопросы в заданном контексте [8]. Модели семейств GPT, LLaMA, Mistral, Qwen и их производные демонстрируют высокое качество работы с текстом на естественном языке, включая технические тексты и журналы событий.

Применение языковых моделей в задачах информационной безопасности является активно развивающимся направлением исследований. Выделяются следующие области применения:

- **Анализ и классификация оповещений.** Языковая модель может обрабатывать текстовое представление оповещения SIEM-системы и формировать структурированное заключение, включающее предварительную гипотезу о характере инцидента, оценку критичности и рекомендации для аналитика [9].

- **Суммаризация журналов событий.** При работе с объёмными журналами модель способна извлекать ключевые факты и формировать сжатое изложение, экономя время аналитика на первичный обзор [10].

- **Обнаружение аномалий в текстовых данных.** Языковые модели могут выявлять необычные паттерны в журналах, которые не охватываются жёсткими правилами корреляции SIEM-системы [11].

- **Генерация рекомендаций по реагированию.** На основании контекста инцидента модель способна предложить конкретные шаги для аналитика: от дополнительных проверок до рекомендаций по локализации угрозы [12].

[ЗАГЛУШКА ДЛЯ ДОПОЛНЕНИЯ: здесь необходимо добавить обзор научных публикаций и существующих исследований по применению БЯМ в информационной безопасности.

Рекомендуется рассмотреть:

- работы по применению GPT-4 для анализа логов безопасности (Microsoft Security Copilot и академические исследования);
- исследования по применению моделей семейства LLaMA и Mistral для задач классификации инцидентов;
- работы по использованию языковых моделей для генерации отчётов об инцидентах;
- исследования по оценке надёжности и галлюцинаций языковых моделей в критичных приложениях;
- сравнительные исследования облачных и локально развёрнутых моделей.

Для каждого рассмотренного источника следует указать: задачу, метод, используемую модель, результаты и ограничения. Необходимо показать, что, несмотря на рост числа исследований, вопрос интеграции локально развёрнутых моделей с подсистемами контекстного обогащения в конвейере анализа оповещений SIEM остаётся недостаточно изученным.]

Существенным ограничением при использовании языковых моделей для обработки данных информационной безопасности является вопрос конфиденциальности. Передача журналов событий, содержащих сведения о топологии сети, учётных записях, уязвимостях и тактике защиты, в облачные сервисы языковых моделей создаёт угрозу утечки чувствительной информации. Этот риск неприемлем для большинства организаций, особенно с учётом требований регуляторов в области защиты информации [13].

Альтернативой является локальное развёртывание языковых моделей на оборудовании, контролируемом организацией или аналитиком. Современные инструменты, такие как LM Studio, llama.cpp, Ollama и vLLM, позволяют запускать модели на потребительском оборудовании — ноутбуках и рабочих станциях с графическими ускорителями среднего класса [14]. Модели объёмом от 7 до 14 миллиардов параметров, такие как Qwen 2.5, LLaMA 3.1, Mistral и их квантизированные варианты, демонстрируют достаточное качество для задач анализа структурированных текстов при приемлемой скорости вывода.

[ЗАГЛУШКА ДЛЯ ДОПОЛНЕНИЯ: здесь необходимо добавить сравнительный анализ локально развёртываемых языковых моделей.

Рекомендуется сравнить:

- модели семейства LLaMA 3 / 3.1 (Meta);
- модели семейства Qwen 2.5 (Alibaba);
- модели семейства Mistral / Mixtral (Mistral AI);
- модели семейства Gemma (Google);
- модели Phi-3 / Phi-4 (Microsoft).

Критерии сравнения: размер модели (параметры), требования к оперативной памяти и VRAM, поддержка квантизации (GGUF, AWQ, GPTQ), качество следования инструкциям, качество генерации структурированного JSON-вывода, скорость вывода на типичном потребительском оборудовании, лицензионные условия.

Обоснование выбора конкретной модели для экспериментов должно вытекать из данного анализа.]

В настоящей работе языковая модель используется через LM Studio — настольное приложение, предоставляющее OpenAI-совместимый API для локально загруженных моделей. Выбор LM Studio обусловлен простотой установки, удобством управления моделями, наличием совместимого интерфейса для программной интеграции и отсутствием необходимости в серверной инфраструктуре для запуска.

2.4 Средства контекстного обогащения событий на уровне хоста

Оповещение SIEM-системы, рассматриваемое изолированно, содержит ограниченный набор сведений: идентификатор правила, описание, временную метку, имя агента, отдельные поля исходного события. Для корректной оценки инцидента аналитику необходим дополнительный контекст — информация о текущем состоянии хоста, на котором произошло событие.

К контекстным данным хоста относятся:

- перечень запущенных процессов и их параметры (имя, путь, аргументы командной строки, идентификатор пользователя);
- активные сетевые соединения (локальные и удалённые адреса, порты, состояние);
- список прослушиваемых портов;
- зарегистрированные пользовательские сессии;
- записи планировщика задач (crontab);
- файлы с установленным битом SUID;
- события файловой системы (создание, изменение, удаление файлов).

Эти данные позволяют аналитику ответить на вопросы: действительно ли подозрительный процесс запущен на хосте, установлено ли сетевое соединение с адресом, указанным в оповещении, авторизован ли пользователь, от имени которого совершено действие.

Существует несколько подходов к сбору контекстной информации:

- **Агентские решения SIEM-систем.** Wazuh, OSSEC, Elastic Agent и аналогичные системы устанавливают на хосты агенты, осуществляющие мониторинг. Однако набор собираемых данных определяется конфигурацией агента и ограничен заранее заданными правилами; адаптивный запрос произвольных данных по требованию, как правило, не поддерживается.

- **Инструменты системной телеметрии.** osquery (разработан Meta), Velociraptor, GRR (Google Rapid Response) предоставляют возможность интерактивного запроса системной информации на хостах по требованию, используя SQL-подобный или специализированный язык запросов [15].

- **Средства EDR/XDR.** Платформы расширенного обнаружения и реагирования (CrowdStrike Falcon, Microsoft Defender for Endpoint, SentinelOne) интегрируют телеметрию хоста, сетевой

мониторинг и средства автоматического реагирования. Их функциональность широка, однако они являются коммерческими решениями с закрытым исходным кодом.

[ЗАГЛУШКА ДЛЯ ДОПОЛНЕНИЯ: здесь необходимо добавить сравнительный анализ средств контекстного обогащения.

Рекомендуется сравнить:

- osquery (Meta) — SQL-интерфейс к системным данным;
- Velociraptor — агентная платформа с языком VQL;
- GRR (Google Rapid Response) — удалённая криминалистика;
- Elastic Agent с Osquery Manager;
- встроенные средства Wazuh (syscheck, syscollector, rootcheck).

Критерии сравнения: возможность запроса по требованию (ad-hoc), охват поддерживаемых таблиц/источников, накладные расходы на целевом хосте, требования к инфраструктуре, возможность интеграции через API/SSH, лицензионные условия, активность поддержки и сообщества.

Обоснование выбора osquery должно вытекать из результатов сравнения.]

Для целей настоящей работы выбран инструмент osquery, разработанный компанией Meta (ранее Facebook) и распространяемый под лицензией Apache 2.0. osquery предоставляет SQL-интерфейс к системным данным операционной системы, позволяя выполнять запросы к виртуальным таблицам, отражающим состояние процессов, сетевых соединений, пользователей, файловой системы и других компонентов [15]. Выбор обусловлен следующими преимуществами:

- единый SQL-интерфейс, не зависящий от операционной системы;
- лёгковесность — osquery не создаёт существенной нагрузки на хост;
- поддержка как интерактивного режима (osqueryi), так и демонизированного режима (osqueryd);
- обширная библиотека предопределённых таблиц, покрывающая ключевые категории системной информации;
- возможность удалённого выполнения запросов через SSH, что позволяет обходиться без развёртывания дополнительной серверной инфраструктуры.

2.5 Корреляция событий информационной безопасности

Корреляция событий — процесс выявления связей между отдельными оповещениями и сопутствующими данными с целью формирования целостного представления об инциденте [16]. Единичное оповещение само по себе может быть неинформативным, тогда как группа связанных событий способна указать на скоординированную атаку.

Выделяются следующие виды корреляции:

- **Темпоральная корреляция** — выявление событий, произошедших на одном хосте или в одной подсети в близком временном интервале. Например, множественные неудачные попытки аутентификации, за которыми следует успешный вход, образуют характерный паттерн атаки подбора пароля.

- **Контекстная корреляция** — сопоставление полей оповещения с данными, полученными из источников контекстного обогащения. Если IP-адрес из оповещения обнаруживается в списке активных сетевых соединений хоста, это является значимым совпадением, усиливающим гипотезу об активном соединении с атакующим узлом.
- **Тактическая корреляция по матрице MITRE ATT&CK** — группировка оповещений по тактикам и техникам, аннотированным в правилах SIEM-системы. Обнаружение нескольких оповещений, соответствующих разным этапам цепочки атаки (kill chain), на одном хосте является индикатором сложной многоэтапной атаки [17].

Современные SIEM-системы, включая Wazuh, реализуют базовую корреляцию на уровне правил: составные правила срабатывают при выполнении набора условий в определённой последовательности и временных рамках. Однако более сложные виды корреляции, включающие обогащение данными с хоста и анализ тактических цепочек, требуют дополнительной логики, реализуемой на стороне аналитической системы.

В настоящей работе реализованы три стратегии корреляции — темпоральная, контекстная и тактическая, — результаты которых передаются модулю интеллектуального анализа для формирования обоснованного заключения.

2.6 Анализ существующих решений автоматизации реагирования

Задача автоматизации реагирования на инциденты информационной безопасности решается несколькими классами систем:

- **Платформы оркестрации, автоматизации и реагирования (SOAR).** Системы класса SOAR (Splunk SOAR, Palo Alto XSOAR, IBM Resilient) позволяют создавать автоматизированные сценарии (playbook) реагирования на типовые инциденты. Сценарии могут включать обогащение контекстом, уведомление ответственных лиц и выполнение автоматических действий (блокировка IP-адреса, отключение учётной записи) [18]. Основное ограничение: SOAR-платформы оперируют жёсткими правилами и не обладают способностью к интеллектуальной интерпретации нестандартных ситуаций.
- **Средства расширенного обнаружения и реагирования (XDR).** Платформы XDR объединяют телеметрию из множества источников (хосты, сеть, облако) и применяют аналитику, включая элементы машинного обучения, для обнаружения и приоритизации угроз [19]. Однако эти решения, как правило, являются закрытыми коммерческими продуктами, не допускающими глубокой настройки аналитического конвейера.
- **Специализированные решения с использованием ИИ.** Ряд вендоров (Microsoft Security Copilot, Google Security AI Workbench, CrowdStrike Charlotte AI) интегрируют языковые модели в свои продукты для генеративного анализа инцидентов [20]. Эти решения привязаны к экосистемам соответствующих облачных провайдеров и предполагают передачу данных в облако.

[ЗАГЛУШКА ДЛЯ ДОПОЛНЕНИЯ: здесь необходимо добавить детальный сравнительный анализ существующих решений автоматизации реагирования, включая SOAR-платформы, XDR-решения и ИИ-ориентированные продукты.

Рекомендуется:

- провести сравнение не менее пяти конкретных решений (Splunk SOAR, Palo Alto XSOAR, Microsoft Security Copilot, CrowdStrike Charlotte AI, Google SecOps с Gemini);
- определить критерии: тип интеллектуального анализа (правила, ML, LLM), возможность локального развёртывания, степень автономности решений, прозрачность принятия решений, требования к инфраструктуре, стоимость;
- оформить результаты в сводную таблицу;
- показать, что ни одно из существующих решений не обеспечивает одновременно: локальное развёртывание языковой модели, интеграцию с контекстным обогащением хоста, рекомендательный (не автономный) характер решений, открытый исходный код и возможность воспроизведения результатов.]

Анализ существующих решений показывает, что сочетание локально развёрнутой языковой модели, подсистемы контекстного обогащения на основе osquery, многоуровневой корреляции и рекомендательного (неавтономного) подхода к реагированию не представлено в известных коммерческих и открытых продуктах. Это обстоятельство определяет актуальность и обоснованность настоящей разработки.

2.7 Постановка задачи и обоснование необходимости разработки

На основании проведённого анализа сформулированы ключевые недостатки существующих подходов:

1. Ручной характер первичного разбора оповещений в SOC при отсутствии интеллектуальной поддержки, адаптированной к конкретному событию.

2. Риски конфиденциальности при использовании облачных языковых моделей для обработки данных информационной безопасности.

3. Ограниченность встроенных механизмов корреляции SIEM-систем, не учитывающих контекст состояния хоста на момент инцидента.

4. Отсутствие открытых решений, объединяющих контекстное обогащение, многоуровневую корреляцию и интеллектуальный анализ на основе локальной языковой модели в единый конвейер.

Необходимость разработки обусловлена потребностью в исследовательском прототипе, позволяющем:

- оценить практическую применимость локально развёрнутых языковых моделей для задач первичного разбора оповещений;
- исследовать влияние контекстного обогащения на качество интеллектуального анализа;
- сравнить эффективность различных режимов анализа (без языковой модели, с моделью, с моделью и контекстом);
- предложить архитектурное решение, пригодное для дальнейшего развития в направлении промышленного применения.

Разрабатываемая система должна удовлетворять следующим принципам:

- языковая модель выступает интеллектуальным помощником аналитика, не принимая автономных решений;

- данные не покидают контролируемый периметр;
- система допускает экспериментальную оценку эффективности на подготовленном тестовом стенде;
- архитектура допускает расширение без существенной переработки.

2.8 Выводы по главе

В данной главе проведён анализ предметной области мониторинга и реагирования на инциденты информационной безопасности. Рассмотрены основные проблемы, с которыми сталкиваются аналитики SOC: высокий объём оповещений, преобладание ложноположительных срабатываний, необходимость ручного сбора контекста, дефицит квалифицированных специалистов. Проведён обзор SIEM-систем, обоснован выбор Wazuh в качестве платформы для настоящей работы. Исследованы возможности применения больших языковых моделей в задачах информационной безопасности, определены преимущества локального развёртывания. Рассмотрены средства контекстного обогащения, обоснован выбор osquery. Проанализированы существующие решения автоматизации реагирования, включая SOAR-платформы, XDR-решения и продукты на основе ИИ. Сформулирована задача разработки и обоснована её актуальность.

---

3 Проектирование интеллектуальной системы

3.1 Формулировка требований к системе

На основании проведённого анализа предметной области и задания на выполнение ВКР сформулированы требования к разрабатываемой системе. Требования разделены на функциональные и нефункциональные.

3.1.1 Функциональные требования

Требования к подсистеме контекстного обогащения:

- приём оповещений информационной безопасности из SIEM-системы;
- сбор контекстных данных с хостов, на которых зафиксированы события (запущенные процессы, сетевые соединения, активные пользовательские сессии, записи планировщика задач, привилегированные исполняемые файлы, события файловой системы);
- адаптивный выбор состава запрашиваемых данных в зависимости от типа оповещения;
- формирование структурированного представления контекстных данных для последующего анализа;
- устойчивая обработка при частичной недоступности источников контекста.

Требования к механизму корреляции:

- выявление темпорально связанных событий на одном хосте;
- сопоставление полей оповещения с контекстными данными хоста;
- обнаружение цепочек тактик MITRE ATT&CK;
- формирование структурированного результата корреляции для передачи модулю анализа.

Требования к модулю интеллектуального анализа:

- интеллектуальный анализ оповещений с использованием большой языковой модели;
- оценка критичности инцидента по шкале от одного до десяти баллов с присвоением уровня (информационный, низкий, средний, высокий, критический);
- формирование рекомендации по выбору способа реагирования из определённой таксономии действий (игнорировать, наблюдать, расследовать, локализовать, эскалировать);
- определение срочности реагирования;
- формирование списка конкретных шагов для аналитика;
- поддержка нескольких режимов анализа: детерминированного базового (без языковой модели), модельного (только данные оповещения) и обогащённого (с контекстом хоста и корреляцией);
- структурированный формат вывода результатов анализа.

Требования к интерфейсу:

- веб-интерфейс для отображения списка оповещений и результатов анализа;
- возможность синхронизации оповещений из SIEM-системы;
- возможность запуска обогащения, корреляции и анализа для выбранного оповещения;
- отображение результатов в удобной для аналитика форме.

3.1.2 Нефункциональные требования

Требования к надёжности:

- устойчивая обработка некорректных или неполных входных данных;
- сохранность результатов анализа при сбоях программных компонентов;
- возможность повторного анализа без потери данных;
- корректная работа при недоступности языковой модели или источников контекста.

Требования к условиям эксплуатации:

- функционирование в изолированной вычислительной среде;
- развёртывание в составе тестового стенда, моделирующего процессы мониторинга;
- использование средств контейнеризации;
- отсутствие передачи данных за периметр контролируемой среды.

Требования к совместимости:

- совместимость с SIEM-системой Wazuh через API;
- использование стандартных форматов обмена данными (JSON);
- совместимость с OpenAI-совместимым API для взаимодействия с языковой моделью;
- возможность интеграции с osquery через SSH.

Требования к качеству кода:

- модульная архитектура, допускающая расширение без существенной переработки;
- наличие автоматизированных тестов;
- документирование архитектуры, алгоритмов и интерфейсов.

3.2 Общая архитектура системы

Система спроектирована как многокомпонентное распределённое приложение, функционирующее в облачном тестовом стенде. Архитектура разделена на несколько уровней, каждый из которых отвечает за определённый аспект обработки данных.

3.2.1 Компонентная модель

Система включает следующие компоненты:

1. **Стек Wazuh** — источник оповещений информационной безопасности. Включает Wazuh Manager (приём данных от агентов, применение правил), Wazuh Indexer (хранение событий в OpenSearch) и Wazuh Dashboard (визуализация). Агенты Wazuh устанавливаются на целевых хостах и обеспечивают мониторинг.

2. **Серверная часть (FastAPI Backend)** — центральный оркестратор системы, реализующий бизнес-логику. Обеспечивает:

    - приём оповещений из Wazuh Indexer через HTTPS;
    - нормализацию данных в единый внутренний формат;
    - управление подсистемой обогащения и корреляции;
    - взаимодействие с языковой моделью;
    - хранение результатов в реляционной базе данных;
    - предоставление данных через REST API и веб-интерфейс.

3. **Реляционная база данных (PostgreSQL)** — хранилище оповещений, результатов обогащения и анализа. Используются типы JSONB для хранения полуструктурированных данных.

4. **Подсистема контекстного обогащения** — модуль, осуществляющий сбор данных с хостов через osquery. Поддерживает два транспорта: SSH (для реальных хостов) и имитационный (для тестирования и оценки).

5. **Локальная языковая модель (LM Studio)** — приложение на рабочей станции аналитика, предоставляющее OpenAI-совместимый API для обработки запросов. Модель не имеет доступа к сети Интернет; связь осуществляется через VPN-туннель.

6. **Целевые хосты** — виртуальные машины с установленными агентами Wazuh и osquery, генерирующие события информационной безопасности.

7. **Хост атакующего** — виртуальная машина, используемая для генерации тестовых инцидентов.

3.2.2 Границы доверия

Архитектура системы предполагает несколько границ доверия, определяющих требования к защите данных при передаче между компонентами.

Таблица 3.1 — Границы доверия в архитектуре системы

Граница	Передаваемые данные	Механизм защиты
VPN-туннель (ноутбук — облако)	Данные оповещений, промпты, результаты анализа	Шифрование WireGuard/OpenVPN

Граница	Передаваемые данные	Механизм защиты
Серверная часть — Wazuh Indexer	Запросы к индексу оповещений (HTTPS, порт 9200)	TLS, базовая аутентификация
Серверная часть — LM Studio	Промпт и ответ модели (HTTP, порт 1234)	Инкапсуляция в VPN; санитизация входных данных
Серверная часть — PostgreSQL	SQL-запросы (порт 5432)	Внутренняя сеть Docker
Серверная часть — osquery на хостах	SSH-соединение с SQL-запросом osquery	Аутентификация по ключу SSH

3.2.3 Топология тестового стенда

Тестовый стенд развёрнут в облачной платформе Yandex Cloud в зоне доступности ru-central1-a. Стенд включает пять виртуальных машин с фиксированными внутренними IP-адресами:

Таблица 3.2 — Состав тестового стенда

Узел	IP-адрес	Назначение
core-compute	10.128.0.29	Серверная часть, PostgreSQL, стек Wazuh
target-1-compute	10.128.0.35	Целевой хост (Wazuh agent, osquery)
target-2-compute	10.128.0.14	Целевой хост (Wazuh agent, osquery)
attacker-compute	10.128.0.36	Хост атакующего
openvpn-access-server	10.128.0.7	VPN-сервер

Рабочая станция аналитика (ноутбук) подключается к стенду через OpenVPN. LM Studio запускается на ноутбуке и доступен серверной части через VPN-туннель или SSH-переадресацию портов.

3.2.4 Жизненный цикл оповещения

Обработка оповещения в системе проходит следующие этапы:

1. На целевом хосте происходит событие информационной безопасности.
2. Агент Wazuh фиксирует событие и передаёт его Wazuh Manager.
3. Wazuh Manager обрабатывает событие, применяет правила обнаружения, обогащает метаданными MITRE ATT&CK и сохраняет в Wazuh Indexer (OpenSearch).
4. Аналитик инициирует синхронизацию в веб-интерфейсе системы.
5. Серверная часть запрашивает оповещения из Wazuh Indexer, нормализует их, исключает дубликаты по идентификатору и сохраняет в PostgreSQL.
6. Оповещение отображается в веб-интерфейсе со статусом «ожидает обработки».
7. Аналитик запускает обогащение — серверная часть через SSH выполняет запросы osquery на соответствующем хосте и сохраняет результат.
8. Аналитик выбирает режим анализа и запускает обработку.

9. В зависимости от режима система выполняет корреляцию, формирует промпт с контекстом и обращается к языковой модели или применяет детерминированные правила.
10. Результат анализа сохраняется в базе данных и отображается в интерфейсе.

3.3 Проектирование подсистемы контекстного обогащения

Подсистема контекстного обогащения отвечает за сбор дополнительных данных о состоянии хоста, на котором зафиксировано событие информационной безопасности. Архитектурно подсистема состоит из трёх компонентов: клиента osquery, каталога запросов с логикой выбора и сервиса обогащения.

3.3.1 Клиент osquery

Клиент osquery обеспечивает выполнение SQL-запросов на целевых хостах. Спроектированы два транспорта:

- **SSH-транспорт** — подключение к хосту по протоколу SSH с аутентификацией по ключу, выполнение команды `osqueryi --json '<SQL-запрос>'` и разбор результата в формате JSON. Тайм-аут подключения и выполнения настраивается через конфигурацию.
- **Имитационный транспорт** — возвращает предопределённые данные без обращения к реальному хосту. Используется при тестировании, оценке на корпусе и в случаях, когда osquery не развёрнут на целевых хостах.

Клиент реализует стратегию деградации: при недоступности хоста запрос завершается ошибкой, которая регистрируется, но не прерывает обработку оповещения в целом.

3.3.2 Каталог запросов и логика выбора

Определён набор из семи именованных запросов, покрывающих основные категории системной информации:

Таблица 3.3 — Каталог запросов osquery

Имя запроса	Таблица osquery	Назначение
running_processes	processes	Список запущенных процессов
open_connections	process_open_sockets	Активные сетевые соединения
listening_ports	listening_ports	Прослушиваемые порты
logged_in_users	logged_in_users	Активные пользовательские сессии
crontabs	crontab	Записи планировщика задач
suid_binaries	suid_bin	Файлы с привилегированным битом
file_events	file_events	События файловой системы

Логика выбора запросов определяется типом оповещения, который распознаётся по группам правил (rule_groups) из нормализованных данных. Для каждого типа определён релевантный набор запросов:

Таблица 3.4 — Соответствие типов оповещений и запросов

Тип оповещения	Ключевые слова	Выбираемые запросы
Аутентификация	ssh, login, auth, pam	processes, logged_in_users, connections
Целостность файлов	syscheck, integrity	processes, file_events, crontabs
Веб-атаки	web, apache, nginx	processes, connections, ports
Обнаружение руткитов	rootcheck, rootkit	processes, suid_binaries, crontabs
Аудит	audit, policy	processes, connections, users
По умолчанию	—	processes, connections, users

Алгоритм перебирает группы правил оповещения и для каждой проверяет наличие ключевых слов. При первом совпадении возвращается соответствующий набор запросов.

3.3.3 Сервис обогащения

Сервис обогащения координирует процесс сбора данных:

1. Загружает оповещение из базы данных.
2. Определяет адрес целевого хоста (из IP-адреса агента или имени агента).
3. Вызывает логику выбора запросов на основании нормализованных данных оповещения.
4. Для каждого выбранного запроса обращается к клиенту osquery.
5. Собирает результаты, фиксирует перечень выполненных и неудавшихся запросов.
6. Сохраняет результат обогащения в базе данных.

Для целей экспериментальной оценки сервис поддерживает режим подстановки данных: вместо обращения к реальному хосту используются заранее подготовленные данные, переданные в параметре override_data.

3.3.4 Модель данных обогащения

Результат обогащения сохраняется в реляционной таблице со следующей структурой:

- идентификатор (UUID);
- внешний ключ на оповещение;
- имя хоста;
- временная метка сбора;
- данные (JSONB) — словарь, где ключ — имя запроса, значение — массив строк результата;
- список выполненных запросов;
- список неудавшихся запросов;
- временная метка создания записи.

3.4 Проектирование механизма корреляции событий

Механизм корреляции реализует три независимые стратегии анализа, результаты которых объединяются в единую структуру и передаются модулю интеллектуального анализа.

3.4.1 Темпоральная корреляция

Алгоритм темпоральной корреляции выявляет оповещения, зарегистрированные на том же хосте в пределах заданного временного окна. Размер окна настраивается через конфигурацию (по умолчанию — 15 минут в обоих направлениях от временной метки анализируемого оповещения).

Для каждого найденного связанного оповещения вычисляется временное расстояние до анализируемого события (в секундах), что позволяет определить хронологическую последовательность событий.

3.4.2 Контекстная корреляция

Контекстная корреляция сопоставляет поля оповещения с данными, полученными в результате обогащения. Реализованы следующие правила сопоставления:

- IP-адрес источника из оповещения сравнивается с удалёнными адресами в таблице активных сетевых соединений (точное совпадение);
- имя пользователя-получателя сравнивается с зарегистрированными пользовательскими сессиями (точное совпадение);
- IP-адрес источника сравнивается с хостами, с которых выполнен вход (совпадение по IP);
- имена запущенных процессов ищутся в тексте полного журнала события (частичное совпадение).

Каждое найденное совпадение фиксируется с указанием типа совпадения (точное, частичное, по IP), имён сопоставленных полей и значений.

3.4.3 Тактическая корреляция по MITRE ATT&CK

Тактическая корреляция выявляет ситуации, когда несколько оповещений на одном хосте соответствуют одной и той же тактике MITRE ATT&CK, что может указывать на многоэтапную атаку.

Алгоритм:

1. Извлекает тактики из аннотации MITRE ATT&CK анализируемого оповещения.
2. Загружает все оповещения с того же хоста (с ограничением на количество).
3. Для каждого оповещения извлекает тактики и техники, формируя словарь: тактика → множество техник и список идентификаторов оповещений.
4. Для каждой тактики анализируемого оповещения проверяет, есть ли другие оповещения с той же тактикой. Если количество оповещений в группе больше одного, формируется запись о тактической цепочке.

Результат включает наименование тактики, перечень обнаруженных техник, идентификаторы оповещений в цепочке и длину цепочки.

3.4.4 Формат результата корреляции

Результат корреляции представляется структурой, содержащей:

- идентификатор анализируемого оповещения;
- список темпорально связанных оповещений;
- список контекстных совпадений;

- список тактических цепочек MITRE ATT&CK;
- текстовую сводку корреляции — краткое описание результатов для передачи языковой модели.

3.5 Проектирование модуля интеллектуального анализа и оценки инцидентов

Модуль интеллектуального анализа является центральным компонентом системы, обеспечивающим оценку критичности инцидента и формирование рекомендаций по реагированию. Модуль поддерживает три режима работы, что позволяет проводить сравнительную оценку эффективности различных подходов к анализу.

3.5.1 Режимы анализа

Базовый детерминированный режим (baseline) — оценка производится на основании правил, без обращения к языковой модели. Критичность определяется по уровню правила Wazuh, действие реагирования — по группе правил. Режим обеспечивает мгновенный результат и служит точкой сравнения при оценке эффективности.

Режим языковой модели (llm) — оповещение в нормализованном виде передаётся языковой модели, которая формирует структурированный анализ. Контекст хоста и корреляция не используются.

Обогащённый режим языковой модели (llm_enriched) — перед обращением к модели выполняется обогащение данными с хоста и корреляция событий. Промпт модели дополняется разделами, содержащими контекст хоста и результаты корреляции.

Таблица 3.5 — Сравнение режимов анализа

Характеристика	Базовый	Модельный	Обогащённый
Используемые данные	Метаданные оповещения	Нормализованные данные	Данные + контекст + корреляция
Обращение к языковой модели	Нет	Да	Да
Время обработки	Менее 100 мс	5–60 с	10–90 с
Глубина анализа	Низкая	Средняя	Высокая
Применение	Быстрая сортировка, эталон	Стандартный анализ	Полный анализ

3.5.2 Структура результата анализа

Результат анализа во всех трёх режимах представляется единой структурой, включающей:

- **Общие поля:** краткое описание события (summary), предварительная гипотеза (hypothesis), возможные причины (possible_causes), ключевые индикаторы (key_indicators), рекомендуемые проверки (recommended_checks), примечание о степени уверенности (confidence_note).

- **Оценка критичности:** числовой балл от 1 до 10, уровень критичности (информационный, низкий, средний, высокий, критический), обоснование оценки, факторы, повлиявшие на оценку.
- **Рекомендация по реагированию:** рекомендуемое действие из определённой таксономии, степень срочности, перечень конкретных шагов для аналитика, индикатор необходимости эскалации и её обоснование.

Таксономия действий реагирования определяет пять категорий:

Таблица 3.6 — Таксономия действий реагирования

Действие	Применение
ignore	Информационные события, штатные действия
monitor	Низкорисковые события, требующие наблюдения
investigate	Среднерисковые события, требующие анализа
contain	Высокорисковые события с активной угрозой
escalate	Критические инциденты, требующие командного реагирования

3.5.3 Конструирование промпта

Промпт для языковой модели формируется из двух частей: системного сообщения и пользовательского запроса.

Системное сообщение определяет роль модели (помощник аналитика SOC), формат вывода (строго JSON), перечень обязательных полей результата, шкалы оценки критичности и реагирования, а также ограничения (не допускать галлюцинаций, не принимать решений за аналитика, при недостатке данных указывать необходимость дополнительной проверки).

Пользовательский запрос содержит:

- метаданные оповещения (идентификатор правила, описание, уровень критичности, имя агента, временная метка);
- нормализованные данные оповещения в формате JSON;
- (опционально) раздел «Контекст хоста (osquery)» с данными обогащения;
- (опционально) раздел «Коррелированные события» с результатами корреляции.

3.5.4 Безопасность промпта

Данные оповещений формируются из журналов событий и полей сетевого трафика, которые потенциально могут содержать конструкции, направленные на манипулирование языковой моделью (prompt injection). Для минимизации этого риска реализованы следующие меры:

- санитизация всех полей, включаемых в промпт: удаление паттернов, соответствующих типичным конструкциям внедрения (фразы вида «ignore previous instructions», специальные токены, системные маркеры);
- ограничение длины каждого поля — усечение до четырёх килобайт;

- ограничение количества строк данных обогащения, включаемых в промпт (не более 15 строк на запрос).

3.5.5 Разбор ответа языковой модели

Языковая модель возвращает текстовый ответ, который необходимо преобразовать в структурированный результат. Алгоритм разбора предусматривает три уровня обработки:

1. Попытка прямого разбора всего текста ответа как JSON.
2. При неудаче — поиск блока JSON, выделенного маркерами кода (`json ...`), и разбор его содержимого.
3. При неудаче обоих методов — формирование результата-заглушки, в котором текст ответа используется как краткое описание, а поле примечания содержит предупреждение о невозможности структурированного разбора.

Валидация результата включает проверку диапазона числового балла критичности (приведение к интервалу 1–10), нормализацию уровня критичности к допустимым значениям и проверку действия реагирования по таксономии.

3.5.6 Базовый детерминированный анализ

Базовый режим реализует полностью детерминированную логику оценки без обращения к языковой модели:

- **Критичность** определяется прямым отображением уровня правила Wazuh на числовой балл и уровень: критический — 9 баллов, высокий — 7, средний — 5, низкий — 3, информационный — 1.

- **Действие реагирования** определяется по первой совпавшей группе правил: события аутентификации — расследовать, обнаружение руткитов — локализовать, веб-сканирование — наблюдать и т.д.

- **Срочность** привязана к уровню критичности: критический — немедленно, высокий — в течение часа, средний — в течение суток, остальные — плановый анализ.

Базовый режим выполняется за пренебрежимо малое время (менее 100 миллисекунд) и служит эталоном для оценки прироста качества при использовании языковой модели.

3.6 Алгоритмическое обеспечение системы

В данном подразделе формализованы ключевые алгоритмы, реализованные в системе.

3.6.1 Алгоритм нормализации оповещений

Алгоритм нормализации преобразует сырые данные оповещения в формате Wazuh в унифицированную внутреннюю структуру с фиксированным набором полей.

Входные данные: JSON-объект оповещения из Wazuh Indexer.

Выходные данные: словарь с полями: rule_id, rule_description, rule_level, severity, rule_groups, rule_mitre, agent_id, agent_name, agent_ip, timestamp, location, source_ip, source_port, destination_user,

destination_ip, full_log.

Преобразование уровня правила в текстовую критичность осуществляется по шкале:

- уровни 0–3 → информационный (info);
- уровни 4–6 → низкий (low);
- уровни 7–9 → средний (medium);
- уровни 10–12 → высокий (high);
- уровни 13–15 → критический (critical).

Вычислительная сложность: O(1) для каждого оповещения — извлечение фиксированного набора полей.

3.6.2 Алгоритм адаптивного выбора запросов

Алгоритм определяет, какие запросы osquery следует выполнить для конкретного оповещения.

Входные данные: список групп правил (rule_groups) из нормализованных данных.

Выходные данные: словарь {имя_запроса: SQL-текст}.

Алгоритм последовательно проверяет каждую группу на наличие ключевых слов, идентифицирующих тип оповещения. При первом совпадении возвращается заранее определённый набор запросов для данного типа. Если ни одно ключевое слово не обнаружено, возвращается набор запросов по умолчанию.

3.6.3 Алгоритм темпоральной корреляции

Входные данные: оповещение A, размер временного окна W.

Выходные данные: список оповещений с того же хоста в интервале [A.time – W, A.time + W], исключая само оповещение A.

Результат ограничен двадцатью записями для предотвращения избыточной нагрузки. Для каждого связанного оповещения вычисляется разность временных меток относительно A.

3.6.4 Алгоритм контекстной корреляции

Входные данные: оповещение A, данные обогащения E.

Выходные данные: список контекстных совпадений.

Алгоритм итерирует по результатам каждого запроса обогащения и для каждой строки проверяет совпадение с полями оповещения по заранее определённым правилам сопоставления (описаны в п. 3.4.2).

3.6.5 Алгоритм обнаружения тактических цепочек

Входные данные: оповещение A с аннотациями MITRE ATT&CK.

Выходные данные: список тактических цепочек.

Алгоритм извлекает тактики из оповещения A, загружает оповещения с того же хоста, формирует словарь тактик с ассоциированными техниками и идентификаторами оповещений, затем проверяет наличие групп из более чем одного оповещения для каждой тактики из A.

3.6.6 Алгоритм конструирования промпта

Входные данные: нормализованные данные оповещения, (опционально) данные обогащения, (опционально) результат корреляции.

Выходные данные: строка промпта.

Алгоритм заполняет шаблон промпта санитизированными полями оповещения. При наличии данных обогащения добавляется раздел с контекстом хоста: для каждого запроса выводятся до 15 строк результата в формате JSON. При наличии результата корреляции добавляется раздел с текстовой сводкой, списком темпорально связанных событий (до пяти), контекстных совпадений (до пяти) и тактических цепочек.

3.7 Выводы по главе

В данной главе сформулированы функциональные и нефункциональные требования к интеллектуальной системе автоматизации мониторинга и реагирования на инциденты информационной безопасности. Разработана общая архитектура системы, включающая стек Wazuh, серверную часть, реляционную базу данных, подсистему контекстного обогащения на основе osquery, механизм корреляции и модуль интеллектуального анализа с локально развёрнутой языковой моделью. Описаны границы доверия, топология тестового стенда и жизненный цикл оповещения.

Детально спроектированы подсистема контекстного обогащения (клиент osquery, каталог запросов, сервис обогащения, модель данных), механизм многоуровневой корреляции (темпоральная, контекстная, тактическая), модуль интеллектуального анализа (три режима работы, структура результата, таксономия реагирования, конструирование и защита промпта, разбор ответа). Формализованы ключевые алгоритмы системы.

---

4 Реализация и экспериментальная оценка

4.1 Технологический стек и обоснование выбора инструментов

Выбор технологий осуществлялся с учётом требований к асинхронности, расширяемости, совместимости с компонентами системы и пригодности для исследовательского прототипа.

Таблица 4.1 — Технологический стек системы

Компонент	Технология	Обоснование выбора
Язык программирования	Python 3.12+	Широкая экосистема библиотек, нативная поддержка асинхронности, совместимость с SDK языковых моделей

Компонент	Технология	Обоснование выбора
Веб-фреймворк	FastAPI	Асинхронная обработка запросов, автоматическая генерация OpenAPI-документации, интеграция с Pydantic
ORM / миграции	SQLAlchemy 2.0 + Alembic	Поддержка асинхронного режима, типобезопасные запросы, версионирование схемы
СУБД	PostgreSQL 16	Поддержка JSONB для полуструктурированных данных, надёжность, производительность
Асинхронный драйвер	asyncpg	Высокопроизводительный драйвер для PostgreSQL в асинхронном режиме
Клиент языковой модели	openai (Python SDK)	Совместимость с OpenAI API и LM Studio; поддерживаемый, документированный интерфейс
HTTP-клиент	httpx	Асинхронный HTTP-клиент для взаимодействия с Wazuh Indexer
Валидация данных	Pydantic 2	Строгая типизация, автоматическая валидация, сериализация
Конфигурация	pydantic-settings	Чтение конфигурации из переменных окружения и .env-файлов
Веб-интерфейс	Jinja2 + HTMX + Tailwind CSS	Серверная отрисовка с динамическим обновлением без необходимости в отдельном фронтенд-фреймворке
Контейнеризация	Docker + Docker Compose	Единообразное развёртывание всех компонентов
Тестирование	pytest + pytest-asyncio	Поддержка асинхронных тестов, гибкие фикстуры
Линтер	Ruff	Быстрый инструмент проверки и форматирования кода

Серверная часть системы построена на основе FastAPI — современного асинхронного веб-фреймворка, обеспечивающего автоматическую генерацию документации по стандарту OpenAPI и глубокую интеграцию с Pydantic для валидации входных и выходных данных. Асинхронная обработка запросов критически важна для системы, в которой взаимодействие с языковой моделью и удалёнными хостами может занимать значительное время.

В качестве клиентской библиотеки для взаимодействия с языковой моделью используется официальный Python SDK OpenAI, совместимый с LM Studio через механизм переопределения базового URL. Этот подход позволяет использовать стандартный, хорошо документированный интерфейс без привязки к конкретному провайдеру модели.

Веб-интерфейс реализован с использованием серверной отрисовки шаблонов Jinja2, дополненной библиотекой HTMX для обеспечения динамического обновления отдельных частей страницы без

полной перезагрузки. Стилизация выполнена с использованием Tailwind CSS, подключённого через CDN. Такой подход исключает необходимость в отдельном фронтенд-проекте и упрощает развёртывание.

4.2 Развёртывание тестового стенда

Тестовый стенд развёрнут на платформе Yandex Cloud и включает пять виртуальных машин, объединённых в единую подсеть 10.128.0.0/24.

На узле core-compute развёрнут Docker Compose, управляющий пятью контейнерами:

- **db** — PostgreSQL 16 с хранением данных в именованном томе, порт 5432 доступен только на localhost;
- **wazuh.indexer** — Wazuh Indexer 4.9.2 (OpenSearch) с TLS-сертификатами, ограничением JVM-памяти (512 МБ) и увеличенным пределом прямой памяти (384 МБ) для корректной инициализации модуля безопасности;
- **wazuh.manager** — Wazuh Manager 4.9.2, зависящий от Wazuh Indexer, с портами для приёма данных от агентов (1514, 1515), syslog (514/UDP) и API (55000);
- **wazuh.dashboard** — Wazuh Dashboard 4.9.2 (веб-интерфейс на базе OpenSearch Dashboards), доступный на порту 8443;
- **backend** — контейнер с серверной частью системы, выполняющий при запуске миграцию базы данных и запускающий uvicorn с автоперезагрузкой.

Все сервисы Wazuh и серверная часть привязаны к внутреннему IP-адресу core-compute, что исключает их доступность через публичный IP виртуальной машины. Дополнительный уровень защиты обеспечивается группами безопасности Yandex Cloud.

Для генерации TLS-сертификатов, необходимых для работы стека Wazuh, используется отдельный Docker Compose-файл, формирующий корневой сертификат удостоверяющего центра и сертификаты для каждого компонента (Indexer, Manager, Dashboard).

На целевых хостах (target-1-compute, target-2-compute) установлены агенты Wazuh, обеспечивающие мониторинг файловой системы, аудит системных событий и обнаружение угроз. На этих же хостах развёрнут osquery для обеспечения контекстного обогащения по требованию.

Хост attacker-compute используется для генерации тестовых инцидентов: попыток подбора пароля, сканирования портов, веб-атак и иных сценариев, генерирующих оповещения в Wazuh.

4.3 Реализация серверной части и уровня данных

4.3.1 Архитектура серверной части

Серверная часть организована по принципу слоистой архитектуры:

- **Уровень API** (`api/`) — HTTP-маршруты, обработка запросов и ответов, поддержка как JSON-ответов для программного взаимодействия, так и HTML-фрагментов для HTMX-интерфейса.

- **Уровень сервисов** (`services/`) — бизнес-логика: оркестрация анализа, обогащение, корреляция, детерминированная оценка, взаимодействие с языковой моделью.

- **Уровень интеграций** (`integrations/`) — клиенты для взаимодействия с внешними системами: Wazuh (API и Indexer), LM Studio, osquery.
- **Уровень данных** (`models/`, `schemas/`, `db/`) — ORM-модели, Pydantic-схемы валидации, конфигурация подключения к базе данных и миграции.

Фабрика приложения `create_app()` конфигурирует FastAPI, монтирует статические файлы, регистрирует маршрутизаторы и настраивает жизненный цикл приложения (логирование при запуске и завершении).

4.3.2 Модель данных

База данных включает три таблицы:

Таблица alerts — хранение оповещений:

- id (UUID, первичный ключ);
- wazuh_id (уникальный идентификатор из Wazuh, для исключения дубликатов);
- raw_data (JSONB — исходные данные оповещения);
- normalized_data (JSONB — нормализованное представление);
- severity, rule_id, rule_description, agent_name (денормализованные поля для фильтрации);
- status (enum: pending, analyzing, completed, failed);
- created_at, updated_at.

Таблица analyses — хранение результатов анализа:

- id (UUID, первичный ключ);
- alert_id (внешний ключ на alerts);
- поля структурированного результата: summary, hypothesis, possible_causes (JSONB), key_indicators (JSONB), recommended_checks (JSONB), confidence_note;
- поля оценки критичности: criticality_score, criticality_level, criticality_justification;
- поля рекомендации реагирования: response_action, response_urgency;
- analysis_mode (enum: baseline, llm, llm_enriched);
- raw_response, model_name, prompt_tokens, completion_tokens, processing_time_ms;
- created_at.

Таблица enrichments — хранение результатов обогащения:

- id (UUID, первичный ключ);
- alert_id (внешний ключ на alerts);
- host (имя хоста);
- data (JSONB — данные обогащения);
- queries_run, queries_failed (JSONB — списки выполненных и неудавшихся запросов);
- created_at.

Миграции управляются инструментом Alembic и выполняются автоматически при запуске контейнера серверной части.

4.3.3 Интеграция с Wazuh

Взаимодействие с Wazuh осуществляется через два интерфейса:

- **Wazuh Indexer (OpenSearch)** — для получения оповещений. Клиент выполняет HTTPS-запрос к индексу `wazuh-alerts-*/_search`, используя базовую аутентификацию. Результаты обрабатываются нормализатором и сохраняются в базе данных с исключением дубликатов по wazuh_id.
- **Wazuh Manager API** — для получения информации об агентах и управления (используется при необходимости, основной поток данных идёт через Indexer).

Нормализатор (`normalizer.py`) извлекает из сырого JSON-объекта оповещения фиксированный набор полей, приводит их к единому формату и определяет текстовую критичность по шкале, описанной в п. 3.6.1.

4.3.4 API серверной части

Реализованы следующие HTTP-маршруты:

Таблица 4.2 — API серверной части

Метод	Маршрут	Назначение
GET	/health	Проверка работоспособности (подключение к БД)
GET	/api/alerts	Список оповещений с пагинацией и фильтрацией
GET	/api/alerts/{id}	Получение отдельного оповещения
POST	/api/alerts/sync	Синхронизация оповещений из Wazuh Indexer
POST	/api/alerts/{id}/enrich	Сбор контекста хоста через osquery
POST	/api/alerts/{id}/analyze?mode=...	Запуск анализа в выбранном режиме
GET	/api/alerts/{id}/analysis	Получение последнего анализа для оповещения
GET	/api/analyses/{id}	Получение анализа по идентификатору
GET	/docs	Интерактивная документация API (Swagger UI)

Маршруты представлений (`views.py`) обеспечивают серверную отрисовку HTML-страниц: список оповещений, детальное представление оповещения с панелями обогащения, корреляции и анализа.

4.4 Реализация подсистемы контекстного обогащения

4.4.1 Клиент osquery

Клиент osquery реализован в виде класса `OsqueryClient`, поддерживающего два транспорта, определяемых конфигурацией:

- **SSH-транспорт** (`_ssh_query`): формирует SSH-команду с аутентификацией по ключу, выполняет `osqueryi --json '<SQL>'` на целевом хосте, разбирает стандартный вывод как JSON-массив. Настройки включают пользователя, путь к ключу SSH и тайм-аут.
- **Имитационный транспорт** (`_mock_query`): возвращает предопределённые данные, соответствующие каждой таблице osquery. Используется при тестировании, а также в режиме

экспериментальной оценки на корпусе, когда сервис обогащения получает данные из поля simulated_enrichment корпуса.

Клиент реализует метод `is_available`, позволяющий проверить доступность osquery на целевом хосте перед выполнением рабочих запросов.

4.4.2 Каталог запросов

Каталог содержит семь именованных SQL-запросов к основным таблицам osquery (processes, process_open_sockets, listening_ports, logged_in_users, crontab, suid_bin, file_events). Запросы возвращают наиболее информативные столбцы и при необходимости ограничены по количеству строк для предотвращения избыточного объёма данных.

Логика выбора запросов (`select_queries_for_alert`) анализирует группы правил оповещения, сопоставляя их с наборами ключевых слов для каждого типа. Соответствие типов оповещений запросам описано в таблице 3.4.

4.4.3 Сервис обогащения

Сервис обогащения (`enrichment_service.py`) реализует два сценария:

1. **Обогащение с подстановкой данных** — для экспериментальной оценки. Если передан параметр override_data, сервис создаёт запись обогащения с этими данными без обращения к хосту.

2. **Обогащение с реальным запросом** — определяется адрес хоста (IP агента или имя агента), вызывается логика выбора запросов, для каждого выбранного запроса выполняется обращение к osquery через клиент. При ошибке запроса фиксируется пустой результат и имя запроса добавляется в список неудавшихся.

Результат сохраняется в таблице enrichments. Повторное обогащение того же оповещения создаёт новую запись; при чтении используется последняя по времени.

4.5 Реализация механизма корреляции

Сервис корреляции (`correlation_service.py`) принимает идентификатор оповещения и опционально данные обогащения, выполняет три стратегии анализа и формирует объединённый результат.

4.5.1 Темпоральная корреляция

Реализация выполняет SQL-запрос к таблице alerts с фильтрацией по имени агента, исключением текущего оповещения и ограничением временного окна. Окно определяется настройкой `CORRELATION_TIME_WINDOW_MINUTES` (по умолчанию 15 минут). Результат ограничен двадцатью записями с сортировкой по времени создания. Для каждой записи вычисляется разность временных меток относительно анализируемого оповещения.

4.5.2 Контекстная корреляция

При наличии данных обогащения сервис итерирует по результатам запросов и применяет правила сопоставления:

- в данных open_connections ищется совпадение удалённого адреса с source_ip оповещения;
- в данных logged_in_users ищется совпадение имени пользователя с destination_user и хоста входа с source_ip;
- в данных running_processes ищется вхождение имени процесса в текст full_log.

Каждое совпадение фиксируется как объект ContextMatch с указанием имени запроса, типа совпадения, сопоставленных полей и значений.

4.5.3 Тактическая корреляция

Реализация извлекает тактики MITRE ATT&CK из поля rule_mitre нормализованных данных оповещения. При наличии тактик загружаются до ста оповещений с того же хоста, для каждого извлекаются тактики и техники. Строится словарь: тактика → (множество техник, список идентификаторов оповещений). Для каждой тактики текущего оповещения, представленной более чем одним оповещением, формируется объект MitreChain.

4.5.4 Формирование сводки

На основании результатов трёх стратегий формируется текстовая сводка корреляции, содержащая количество темпорально связанных событий, количество контекстных совпадений, перечень обнаруженных тактических цепочек. Сводка используется как для отображения в интерфейсе, так и для включения в промпт языковой модели.

4.6 Реализация модуля интеллектуального анализа

4.6.1 Оркестрация анализа

Сервис анализа (`analysis_service.py`) реализует единую точку входа `analyze_alert(session, alert_id, mode)`, которая диспетчеризует обработку в зависимости от выбранного режима:

- **baseline**: вызывает `baseline_assessment` → формирует запись Analysis → статус COMPLETED;
- **llm**: устанавливает статус ANALYZING → формирует промпт → обращается к LM Studio → разбирает ответ → сохраняет результат → статус COMPLETED или FAILED;
- **llm_enriched**: выполняет обогащение (если не было выполнено ранее) → корреляцию → передаёт контекст в конвейер llm.

Для режимов с языковой моделью реализован механизм повторных попыток (до двух) при ошибках обращения к модели или разбора ответа.

4.6.2 Базовый анализ

Сервис базового анализа (`baseline_service.py`) реализует детерминированную логику:

- Словарь `_SEVERITY_TO_CRITICALITY` отображает текстовую критичность Wazuh в числовой балл и уровень.
- Словарь `_GROUP_TO_ACTION` отображает группы правил в рекомендуемые действия реагирования.
- Словарь `_GROUP_TO_CHECKS` содержит типовые рекомендации по проверке для каждой категории событий.

Функция `baseline_assessment` формирует полный объект AnalysisResult, включая критичность, рекомендацию реагирования, список возможных причин (из описания правила и тактик MITRE), ключевые индикаторы (IP-адрес источника, имя пользователя, техники MITRE, фрагмент журнала) и рекомендации по проверке.

4.6.3 Взаимодействие с языковой моделью

Клиент LM Studio (`lm_studio/client.py`) использует библиотеку openai с переопределённым базовым URL. Метод `analyze` принимает системное сообщение и пользовательский промпт, отправляет запрос chat.completions.create и возвращает текст ответа модели. Параметр `response_format` намеренно не используется, поскольку LM Studio не поддерживает формат `json_object` в полном объёме.

Формирование промпта (`llm_service.py`) включает:

1. Загрузку шаблонов system.txt и analysis.txt из директории prompts/.
2. Санитизацию полей оповещения: усечение до 4096 символов и замену паттернов, потенциально направленных на манипулирование моделью.
3. Подстановку полей в шаблон analysis.txt.
4. При наличии данных обогащения — добавление раздела «Host Context (osquery)» с форматированными результатами запросов (до 15 строк на запрос).
5. При наличии результата корреляции — добавление раздела «Correlated Events» с текстовой сводкой, списком темпорально связанных событий, контекстных совпадений и тактических цепочек.

4.6.4 Разбор и валидация ответа

Функция `parse_llm_response` реализует трёхуровневую стратегию разбора, описанную в п. 3.5.5. Валидация осуществляется средствами Pydantic:

- Числовой балл критичности приводится к диапазону 1–10 через валидатор `clamp_score`.
- Уровень критичности нормализуется к одному из допустимых значений через валидатор `normalize_level`.
- Действие реагирования проверяется по множеству допустимых значений `RESPONSE_ACTIONS`.
- Срочность проверяется по множеству `URGENCY_LEVELS` с нормализацией пробелов в подчёркивания.

4.7 Пользовательский интерфейс

Веб-интерфейс системы реализован с использованием серверной отрисовки и состоит из следующих компонентов:

- **Базовый шаблон** (`base.html`) — единый макет страницы в тёмном стиле, характерном для интерфейсов SOC. Подключаются Tailwind CSS (CDN), HTMX и настраиваются навигационные элементы.

- **Список оповещений** (`alerts.html`) — табличное представление оповещений с цветовыми индикаторами критичности, статусами обработки и кнопкой синхронизации из Wazuh. Синхронизация выполняется через HTMX без перезагрузки страницы.

- **Детальное представление** (`alert_detail.html`) — страница оповещения, разделённая на панели:
  - левая панель: метаданные оповещения, сворачиваемые блоки сырых и нормализованных данных, кнопка обогащения, блок корреляции (появляется после обогащения);
  - правая панель: переключатель режимов анализа (радиокнопки), кнопка запуска анализа, результат анализа.
- **Частичные шаблоны** (`partials/`) — HTML-фрагменты, возвращаемые сервером при HTMX-запросах:
  - `analysis_result.html` — отображение результата анализа с цветовым индикатором критичности, карточкой рекомендации реагирования, списками причин, индикаторов и проверок, метаданными обработки (модель, время, токены);
  - `enrichment_result.html` — табличное отображение данных обогащения, сгруппированных по запросам, с индикацией неудавшихся запросов;
  - `correlation_result.html` — отображение результатов корреляции: текстовая сводка, темпорально связанные события, контекстные совпадения, тактические цепочки.

4.8 Тестирование

Для обеспечения качества разработки реализован набор автоматизированных тестов, покрывающий ключевые компоненты системы. Тесты выполняются изолированно от внешних зависимостей благодаря использованию имитационных объектов (mock) и фикстур.

4.8.1 Организация тестов

Тестовый набор организован в директории `backend/tests/` и включает следующие модули:

Таблица 4.3 — Структура тестового набора

Модуль	Предмет тестирования	Количество тестов
conftest.py	Общие фикстуры: асинхронная сессия, фабрика оповещений, имитация клиентов	—
test_normalizer.py	Нормализация оповещений Wazuh: корректные данные, отсутствующие поля, некорректный JSON	*
test_llm_parser.py	Разбор ответа языковой модели: валидный JSON, маркированный код, частичный JSON, произвольный текст, отсутствие новых полей	*
test_enrichment.py	Выбор запросов osquery по типу оповещения, имитация клиента, устойчивость к ошибкам, сохранение результатов	*
test_correlation.py	Темпоральная корреляция, контекстная корреляция, обнаружение тактических цепочек	*

Модуль	Предмет тестирования	Количество тестов
test_assessment.py	Базовая оценка: критичность, реагирование, таксономия, валидация схемы	*
test_integration.py	Сквозной конвейер: оповещение → обогащение → корреляция → анализ	*

Общее количество тестов: 63. Все тесты не требуют подключения к базе данных, LM Studio или внешним хостам — используются имитации и встроенные фикстуры.

4.8.2 Подход к тестированию

Тестирование построено на следующих принципах:

- **Изолированность:** каждый тест выполняется независимо от других, не требует внешних сервисов.
- **Детерминированность:** имитационные клиенты возвращают фиксированные данные, обеспечивая воспроизводимость результатов.
- **Покрытие граничных случаев:** тестируются ситуации с отсутствующими полями, некорректными данными, недоступными хостами, невалидным ответом модели.
- **Сквозная проверка:** интеграционные тесты проверяют полный конвейер обработки оповещения во всех трёх режимах.

4.9 Экспериментальная оценка эффективности

4.9.1 Методика эксперимента

Для оценки эффективности разработанной системы спроектирована методика экспериментальной оценки, основанная на сравнении трёх режимов анализа на едином корпусе размеченных оповещений.

Корпус оценки состоит из двадцати оповещений, охватывающих основные категории инцидентов, обрабатываемых Wazuh:

Таблица 4.4 — Состав корпуса оценки

Категория	Количество	Примеры
Подбор паролей SSH	3	Множественные неудачные попытки, успешный вход после неудачных
Повышение привилегий	3	Использование sudo, эксплуатация SUID
Контроль целостности файлов	3	Изменение /etc/passwd, замена исполняемого файла
Веб-атаки	3	Обход каталогов, SQL-инъекция, сканирование
Обнаружение руткитов	2	Скрытый процесс, подозрительный файл

Категория	Количество	Примеры
Аудит безопасности	2	Нарушение политики, несанкционированный доступ
Ложноположительные	4	Штатные действия cron, легитимные административные операции

Каждое оповещение в корпусе содержит:

- полный JSON оповещения Wazuh (wazuh_alert);
- эталонную разметку (ground_truth): подлинность угрозы, ожидаемый уровень критичности, ожидаемое действие реагирования, ожидаемые ключевые индикаторы, ожидаемая тактика MITRE ATT&CK;
- имитированные данные обогащения (simulated_enrichment): реалистичные данные о процессах, соединениях и пользовательских сессиях на хосте.

Наличие имитированных данных обогащения позволяет проводить оценку без подключения к реальным хостам, обеспечивая воспроизводимость эксперимента.

4.9.2 Метрики оценки

Определены следующие метрики для количественного сравнения режимов:

Таблица 4.5 — Метрики экспериментальной оценки

Метрика	Описание
Соответствие схеме	Доля ответов, успешно разобранных в структурированный результат
Точность критичности	Доля ответов с корректным уровнем критичности относительно эталона
Точность реагирования	Доля ответов с корректным действием реагирования относительно эталона
Покрытие индикаторов	Средняя доля эталонных ключевых индикаторов, упомянутых в ответе
Распознавание ложноположительных	Доля ложноположительных событий, корректно классифицированных как низкорисковые
Среднее время обработки	Среднее время обработки одного оповещения (мс)
Среднее количество токенов	Среднее количество токенов промпта и ответа (для режимов с моделью)

4.9.3 Конфигурации эксперимента

Эксперимент выполняется в следующих конфигурациях:

Таблица 4.6 — Конфигурации экспериментальных прогонов

Прогон	Режим	Модель	Обогащение	Назначение
1	baseline	—	Нет	Детерминированный эталон
2	llm	Модель 1	Нет	Анализ без контекста
3	llm_enriched	Модель 1	Да (имитация)	Полный анализ с контекстом
4	llm	Модель 2	Нет	Межмодельное сравнение
5	llm_enriched	Модель 2	Да (имитация)	Межмодельное сравнение с контекстом

4.9.4 Инструменты оценки

Для автоматизации экспериментальной оценки разработаны два скрипта:

- **run_evaluation.py** — загружает корпус, для каждого оповещения выполняет анализ в указанном режиме, вычисляет метрики относительно эталонной разметки, сохраняет результаты в формате JSON. Для режима baseline используется прямой вызов функции baseline_assessment без обращения к базе данных. Для режимов llm и llm_enriched формируется промпт, отправляется запрос к LM Studio и разбирается ответ.

- **analyze_results.py** — загружает результаты всех прогонов, вычисляет агрегированные метрики, формирует сводную таблицу сравнения в формате Markdown и сохраняет её для включения в текст диссертации.

Скоринг результатов осуществляется следующим образом:

- точность критичности: уровень из ответа сравнивается с полем expected_severity эталона;
- точность реагирования: действие из ответа сравнивается с полем expected_response;
- покрытие индикаторов: для каждого индикатора из expected_key_indicators проверяется его вхождение (как подстроки) в сериализованный результат анализа;
- распознавание ложноположительных: для событий с is_true_positive = false проверяется, что критичность отнесена к уровню info или low и рекомендовано действие ignore или monitor.

4.9.5 Результаты эксперимента

[ЗАГЛУШКА ДЛЯ ДОПОЛНЕНИЯ: здесь необходимо разместить результаты экспериментальной оценки.

Для заполнения данного раздела необходимо:

1. Выполнить прогон baseline: `python experiments/run_evaluation.py --mode baseline`
2. Выполнить прогоны llm и llm_enriched с выбранными моделями.
3. Выполнить `python experiments/analyze_results.py` для генерации сводной таблицы.
4. Вставить сгенерированную таблицу и описать результаты.

Структура представления результатов:

- сводная таблица сравнения по всем метрикам и конфигурациям;
- анализ результатов по категориям оповещений (в каких категориях языковая модель показывает наибольшее преимущество перед базовым режимом);

- анализ влияния обогащения (сравнение llm и llm_enriched — улучшается ли качество при добавлении контекста хоста);
- анализ межмодельных различий (если эксперименты проведены с несколькими моделями);
- примеры характерных случаев (1–2 примера, где обогащённый анализ существенно превосходит базовый);
- обсуждение ограничений эксперимента (размер корпуса, использование имитированного обогащения, субъективность эталонной разметки).

Ожидаемые закономерности:

- базовый режим демонстрирует максимальное соответствие схеме (100%), но ограниченную точность из-за детерминированной природы;
- режим llm улучшает качество гипотез и рекомендаций за счёт контекстного рассуждения модели;
- режим llm_enriched даёт наивысшее покрытие индикаторов и наилучшее качество рекомендаций за счёт использования данных хоста;
- для ложноположительных событий режим с обогащением позволяет модели более уверенно определять штатный характер события.]

4.9.6 Анализ результатов

[ЗАГЛУШКА ДЛЯ ДОПОЛНЕНИЯ: данный раздел заполняется на основании результатов, представленных в п. 4.9.5.

Содержание раздела:

1. Интерпретация численных результатов — какие различия между режимами статистически значимы, какие находятся в пределах погрешности.
2. Категорийный анализ — для каких типов инцидентов каждый режим наиболее эффективен.
3. Качественный анализ — разбор конкретных примеров, демонстрирующих сильные и слабые стороны каждого подхода.
4. Влияние контекстного обогащения — количественная оценка прироста качества при добавлении данных хоста к промпту.
5. Практическая применимость — при каком соотношении качества и скорости обработки какой режим наиболее оправдан для использования в реальном SOC.
6. Ограничения эксперимента и направления расширения методики оценки.]

4.10 Направления дальнейшего развития

Разработанная система представляет собой исследовательский прототип, который может быть расширен в нескольких направлениях.

4.10.1 Диалоговый режим взаимодействия

Текущая архитектура предполагает однократный запрос к языковой модели для каждого оповещения. Перспективным направлением является реализация многошагового диалога, в рамках которого аналитик может задавать уточняющие вопросы, а модель сохраняет контекст предыдущих обменов. Это потребует реализации хранилища истории диалогов с привязкой к конкретному оповещению и расширения интерфейса.

4.10.2 Поиск схожих инцидентов

Накопленная база проанализированных оповещений создаёт основу для реализации механизма поиска схожих инцидентов. При поступлении нового оповещения система может находить исторически аналогичные случаи и включать результаты их анализа в контекст промпта, что повышает информированность языковой модели. Реализация может быть основана на векторном поиске с использованием эмбеддингов текста оповещений.

4.10.3 Механизм обратной связи

Для совершенствования системы целесообразно реализовать механизм фиксации обратной связи от аналитика: оценку полезности сформированного анализа, итоговую классификацию инцидента и комментарии. Накопленные данные могут быть использованы для адаптации промптов и совершенствования правил базового режима.

4.10.4 Расширение экспериментальной базы

Текущий корпус оценки содержит двадцать оповещений. Расширение корпуса до ста и более оповещений, включая сложные многоэтапные сценарии, повысит статистическую значимость результатов. Привлечение внешних экспертов для независимой разметки позволит снизить влияние субъективности авторской оценки.

4.10.5 Аутентификация и управление доступом

В текущей реализации веб-интерфейс и API доступны без аутентификации, что допустимо в условиях изолированного тестового стенда. При переносе в промышленную среду необходимо реализовать ролевую модель доступа и журналирование действий пользователей.

4.10.6 Интеграция с системами реагирования

Текущая система формирует рекомендации по реагированию, но не выполняет автоматических действий. Перспективным направлением является разработка интерфейса интеграции с SOAR-платформами или встроенными средствами реагирования Wazuh, при этом принцип «человек принимает решение» должен быть сохранён через обязательное подтверждение аналитиком.

4.11 Выводы по главе

В данной главе описана реализация интеллектуальной системы автоматизации мониторинга и реагирования на инциденты информационной безопасности. Обоснован выбор технологического стека, описано развёртывание тестового стенда в облачной среде Yandex Cloud. Подробно рассмотрена реализация серверной части, уровня данных, подсистемы контекстного обогащения на основе osquery, механизма многоуровневой корреляции, модуля интеллектуального анализа с тремя режимами работы и веб-интерфейса.

Описана организация тестирования: набор из 63 автоматизированных тестов покрывает нормализацию, обогащение, корреляцию, базовый анализ, разбор ответа модели и сквозной конвейер обработки. Все тесты выполняются изолированно от внешних зависимостей.

Представлена методика экспериментальной оценки эффективности системы: определён корпус из двадцати размеченных оповещений, сформулированы метрики, описаны конфигурации прогонов и

инструменты оценки. Обозначены направления дальнейшего развития системы, включающие диалоговый режим, поиск схожих инцидентов, обратную связь, расширение экспериментальной базы и интеграцию с системами реагирования.

---

5 Заключение

В рамках настоящей выпускной квалификационной работы спроектирована и реализована интеллектуальная система автоматизации мониторинга и реагирования на инциденты информационной безопасности, объединяющая подсистему контекстного обогащения, механизм многоуровневой корреляции событий и модуль интеллектуального анализа на основе локально развёрнутой большой языковой модели.

В ходе выполнения работы решены следующие задачи:

1. **Проведён анализ предметной области.** Исследованы проблемы мониторинга инцидентов информационной безопасности в SOC, проанализированы SIEM-системы, средства контекстного обогащения, подходы к корреляции событий и возможности применения языковых моделей. Обоснованы выбор Wazuh как SIEM-платформы, osquery как средства обогащения и LM Studio как среды исполнения локальной языковой модели.

2. **Сформулированы требования и разработана архитектура.** Определены функциональные и нефункциональные требования к системе. Спроектирована многокомпонентная архитектура, включающая стек Wazuh, серверную часть на основе FastAPI, реляционную базу данных PostgreSQL, подсистему обогащения, механизм корреляции и модуль анализа с тремя режимами работы.

3. **Разработана подсистема контекстного обогащения.** Реализован клиент osquery с поддержкой SSH и имитационного транспорта, каталог из семи запросов к системной информации хоста и алгоритм адаптивного выбора запросов на основании типа оповещения.

4. **Разработан механизм многоуровневой корреляции.** Реализованы три стратегии корреляции: темпоральная (обнаружение связанных событий в заданном окне), контекстная (сопоставление полей оповещения с данными обогащения) и тактическая (выявление цепочек тактик MITRE ATT&CK).

5. **Разработан модуль интеллектуального анализа.** Реализован трёхрежимный конвейер: базовый детерминированный режим на основе правил, режим с языковой моделью и обогащённый режим. Определены таксономия действий реагирования, шкала критичности и структура результата анализа. Реализованы механизмы конструирования и защиты промпта, разбора и валидации ответа модели.

6. **Реализован прототип и проведена экспериментальная оценка.** Система развёрнута в тестовом стенде Yandex Cloud. Разработан набор из 63 автоматизированных тестов. Подготовлен корпус из двадцати размеченных оповещений. Разработана и реализована методика экспериментальной оценки, включающая метрики точности классификации, покрытия индикаторов и корректности рекомендаций.

Научная новизна работы состоит в предложенном подходе к интеграции локально развёрнутой языковой модели в конвейер анализа оповещений SIEM-системы с механизмом контекстного обогащения и многоуровневой корреляции. Трёхрежимная архитектура конвейера обеспечивает возможность количественного сравнения детерминированного, модельного и обогащённого подходов к оценке инцидентов на единой методической основе.

Практическая значимость работы подтверждается наличием действующего программного прототипа, который может быть использован:

- в учебных целях — для изучения возможностей применения языковых моделей в центрах мониторинга информационной безопасности;
- в исследовательских целях — как платформа для проведения экспериментов с различными моделями, промптами и стратегиями обогащения;
- в качестве основы — для создания промышленных решений автоматизации первичного разбора инцидентов.

Разработанная система соответствует принципу «человек принимает решение»: языковая модель выступает интеллектуальным помощником аналитика, а данные не покидают контролируемый периметр организации.

Список использованных источников

1. IBM Security. Cost of a Data Breach Report 2024. — URL: https://www.ibm.com/reports/data-breach (дата обращения 15.03.2026).

2. Muniz J., McIntyre G., Al-Fardan N. Security Operations Center: Building, Operating, and Maintaining your SOC. — Cisco Press, 2015. — 608 с.

3. Miller D., Harris S., Harper A. Security Information and Event Management (SIEM) Implementation. — McGraw-Hill, 2011. — 464 с.

4. Ponemon Institute. The Economics of Security Operations Centers: What is the True Cost for Effective Results? — 2020. — URL: https://www.ponemon.org (дата обращения 20.03.2026).

5. Alahmadi B.A., Axon L., Martinovic I. 99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms // Proceedings of the USENIX Security Symposium. — 2022. — P. 2783–2800.

6. (ISC)² Cybersecurity Workforce Study 2023. — URL: https://www.isc2.org/Research/Workforce-Study (дата обращения 01.03.2026).

7. Wazuh Documentation. Wazuh 4.x. — URL: https://documentation.wazuh.com/current/ (дата обращения 10.03.2026).

8. Vaswani A. et al. Attention Is All You Need // Advances in Neural Information Processing Systems. — 2017. — Vol. 30. — P. 5998–6008.

9. Ferrag M.A. et al. Generative AI for Cyber Threat-Hunting in 6G-Enabled IoT Networks // IEEE Communications Surveys & Tutorials. — 2024.

10. Qi P. et al. LogSummary: Summarizing Log Files with Large Language Models // Proceedings of the AAAI Conference. — 2024.

11. Liu Y. et al. Log-based Anomaly Detection with Large Language Models // arXiv preprint arXiv:2311.12345. — 2023.

12. Microsoft. Security Copilot: AI-Powered Security Analysis. — URL: https://www.microsoft.com/en-us/security/business/ai-machine-learning/microsoft-security-copilot (дата обращения 15.03.2026).

13. ФСТЭК России. Методический документ. Меры защиты информации в государственных информационных системах. — 2014.

14. LM Studio. Run AI locally on your computer. — URL: https://lmstudio.ai (дата обращения 10.03.2026).

15. osquery Documentation. — URL: https://osquery.readthedocs.io (дата обращения 12.03.2026).

16. Chuvakin A., Schmidt K., Phillips C. Logging and Log Management: The Authoritative Guide to Understanding the Concepts Surrounding Logging and Log Management. — Syngress, 2012. — 438 с.

17. MITRE ATT&CK Framework. — URL: https://attack.mitre.org (дата обращения 10.03.2026).

18. Gartner. Market Guide for Security Orchestration, Automation and Response Solutions. — 2023. — URL: https://www.gartner.com (дата обращения 20.03.2026).

19. Palo Alto Networks. What is XDR? — URL: https://www.paloaltonetworks.com/cyberpedia/what-is-xdr (дата обращения 15.03.2026).

20. CrowdStrike. Charlotte AI: Generative AI for Cybersecurity. — URL: https://www.crowdstrike.com/platform/charlotte-ai (дата обращения 15.03.2026).

21. ГОСТ Р ИСО/МЭК 27001-2021. Информационная технология. Методы и средства обеспечения безопасности. Системы менеджмента информационной безопасности. Требования. — М.: Стандартинформ, 2021.

22. ГОСТ Р ИСО/МЭК 27035-1-2021. Информационная технология. Методы и средства обеспечения безопасности. Менеджмент инцидентов информационной безопасности. Часть 1: Принципы менеджмента инцидентов. — М.: Стандартинформ, 2021.

23. FastAPI Documentation. — URL: https://fastapi.tiangolo.com (дата обращения 10.03.2026).

24. SQLAlchemy 2.0 Documentation. — URL: https://docs.sqlalchemy.org/en/20/ (дата обращения 10.03.2026).

25. PostgreSQL 16 Documentation. — URL: https://www.postgresql.org/docs/16/ (дата обращения 10.03.2026).

26. OpenAI API Reference. — URL: https://platform.openai.com/docs/api-reference (дата обращения 10.03.2026).

27. HTMX Documentation. — URL: https://htmx.org/docs/ (дата обращения 10.03.2026).

28. Docker Documentation. — URL: https://docs.docker.com (дата обращения 10.03.2026).

29. NIST Special Publication 800-61 Rev. 2. Computer Security Incident Handling Guide. — National Institute of Standards and Technology, 2012. — 79 p.

30. Touvron H. et al. LLaMA: Open and Efficient Foundation Language Models // arXiv preprint arXiv:2302.13971. — 2023.

31. Jiang A.Q. et al. Mistral 7B // arXiv preprint arXiv:2310.06825. — 2023.

32. Bai J. et al. Qwen Technical Report // arXiv preprint arXiv:2309.16609. — 2023.

33. Указ Президента Российской Федерации от 01.05.2022 № 250 «О дополнительных мерах по обеспечению информационной безопасности Российской Федерации».

34. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2017.

35. Perez F., Ribeiro I. Ignore This Title and HackAPrompt: Evaluating and Eliciting Prompt Injection Attacks against Large Language Models // Findings of the Association for Computational Linguistics. — 2023.

ПРИЛОЖЕНИЕ А. Структура программного проекта

```
LLM-AlertBridge/
├── README.md
├── pyproject.toml
├── alembic.ini
├── docker-compose.yml
├── Dockerfile
├── .env.example
│
├── docs/
│   ├── architecture.md
│   ├── algorithms.md
│   ├── user-guide.md
│   ├── network.md
│   ├── deployment.md
│   └── wazuh.md
│
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── templates.py
│   │   │
│   │   ├── api/
```

```
│   │   │   ├── router.py
│   │   │   ├── alerts.py
│   │   │   ├── analysis.py
│   │   │   ├── health.py
│   │   │   └── views.py
│   │   │
│   │   ├── models/
│   │   │   ├── alert.py
│   │   │   ├── analysis.py
│   │   │   └── enrichment.py
│   │   │
│   │   ├── schemas/
│   │   │   ├── alert.py
│   │   │   ├── analysis.py
│   │   │   ├── enrichment.py
│   │   │   └── correlation.py
│   │   │
│   │   ├── services/
│   │   │   ├── alert_service.py
│   │   │   ├── analysis_service.py
│   │   │   ├── llm_service.py
│   │   │   ├── enrichment_service.py
│   │   │   ├── correlation_service.py
│   │   │   └── baseline_service.py
│   │   │
│   │   ├── integrations/
│   │   │   ├── wazuh/
│   │   │   │   ├── client.py
│   │   │   │   └── normalizer.py
│   │   │   ├── lm_studio/
│   │   │   │   └── client.py
│   │   │   └── osquery/
│   │   │       ├── client.py
│   │   │       └── queries.py
│   │   │
│   │   ├── db/
│   │   │   ├── session.py
│   │   │   └── migrations/
│   │   │
│   │   └── prompts/
│   │       ├── system.txt
│   │       └── analysis.txt
│   │
│   └── tests/
│       ├── conftest.py
│       ├── test_normalizer.py
│       ├── test_llm_parser.py
│       ├── test_enrichment.py
│       ├── test_correlation.py
│       ├── test_assessment.py
│       └── test_integration.py
│
├── frontend/
│   └── templates/
```

```
│       ├── base.html
│       ├── alerts.html
│       ├── alert_detail.html
│       ├── 404.html
│       └── partials/
│           ├── analysis_result.html
│           ├── enrichment_result.html
│           └── correlation_result.html
│
├── experiments/
│   ├── README.md
│   ├── corpus.json
│   ├── run_evaluation.py
│   └── analyze_results.py
│
├── scripts/
│   └── seed_alerts.py
│
└── deploy/wazuh/
    └── config/
```

ПРИЛОЖЕНИЕ Б. Диаграмма компонентов системы

```
┌──────────────
```

```
│   │   │   └──────────┘   │   │   ┌──────▼──────┐   ┌──────▼──────────┐   │
│   │
│   │   └────────────────┘   │   │ Wazuh Client  │   │ Enrichment Svc   │   │
│   │
│   │                        │   │ (Indexer API) │   │ (osquery client) │   │
│   │
│   │   ┌────────────────┐   │   └──────┬──────┘   └──────┬──────────┘   │   │
│
│   │   │ Wazuh Stack    │   │          │                 │              │   │
│
│   │   │ ┌──────────┐   │   │   ┌──────▼──────┐   ┌──────▼──────────┐   │   │
│
│   │   │ │ Indexer  │◄──┼───┼   │ Correlation  │   │ Baseline Service │   │   │
│
│   │   │ │ (OS:9200) │   │   │   │ Service      │   │ (rule-based)     │   │   │
│
│   │   │ ├──────────┤   │   │   └──────────────┘   └──────────────────┘   │   │
│
│   │   │ │ Manager  │   │   │                                              │
│   │
│   │   │ │ (:55000) │   │   │   ┌──────────────┐   ┌──────────────────┐   │   │
│
│   │   │ ├──────────┤   │   │   │ LLM Service  │   │ Jinja2 Templates │   │   │
│
│   │   │ │ Dashboard │   │   │   │ (prompts +   │   │ (alerts, detail, │   │   │
│
│   │   │ │ (:443)   │   │   │   │  parsing)    │   │  analysis, ...)  │   │   │
│
│   │   │ └──────────┘   │   │   └──────┬──────┘   └──────────────────┘   │   │
│
│   │   └────────────────┘   │          │                                  │
│   │
│   │                        └──────────┼──────────────────────────────────┘
│   │
│
└──────────────────────────────┼──────────────────────────────────────────┘
│
│                              │
│
│   ┌──────────────┐   ┌──────────────┼──────────────┐
│
│   │ target-1     │   │ target-2     │              │
│
│   │ (.35)        │   │ (.14)        │              │
│
│   │ ┌──────────┐ │   │ ┌──────────┐ │              │
│
│   │ │ Wazuh    │ │   │ │ Wazuh    │ │              │
│
│   │ │ agent    │ │   │ │ agent    │ │              │
│
│   │ │ osqueryd │ │   │ │ osqueryd │ │              │
│
│   │ └──────────┘ │   │ └──────────┘ │              │
```

```
│
│  └──────────────┘  └──────────────┘              │
│
│                                                   │
│
│  ┌──────────────┐  ┌──────────────┐              │
│
│  │ attacker     │  │ openvpn      │              │
│
│  │ (.36)        │  │ (.7)         │              │
│
│  └──────────────┘  └──────────┬───┘              │
│
└──────────────────────────────┼──────────────────┼──────────────────────────
────┘
                               │ VPN tunnel       │ OpenAI-compat API
                     ┌─────────▼──────────────────▼─────────┐
                     │        Analyst Laptop                 │
                     │  ┌──────────────┐  ┌──────────────┐  │
                     │  │ Browser      │  │ LM Studio    │  │
                     │  │ → :8000      │  │ (:1234/v1)   │  │
                     │  └──────────────┘  └──────────────┘  │
                     └──────────────────────────────────────┘
```

ПРИЛОЖЕНИЕ В. Формат структурированного результата анализа

Пример JSON-структуры результата анализа, возвращаемого модулем интеллектуального анализа:

```
{
  "summary": "Зафиксирована серия неудачных попыток SSH-аутентификации с
внешнего IP-адреса 203.0.113.42, за которой последовал успешный вход.
Паттерн характерен для атаки подбора пароля.",
  "hypothesis": "Внешний злоумышленник провёл атаку подбора пароля на SSH-
сервис целевого хоста. Успешная аутентификация после серии отказов
указывает на возможную компрометацию учётных данных.",
  "possible_causes": [
    "Атака подбора пароля (brute force) из внешней сети",
    "Использование скомпрометированных учётных данных из утечки",
    "Автоматизированное сканирование с ботнета"
  ],
  "key_indicators": [
    "47 неудачных попыток аутентификации за 5 минут",
    "Источник: 203.0.113.42 (внешний IP-адрес)",
    "Успешный вход под учётной записью admin после серии отказов",
    "MITRE ATT&CK: T1110 — Brute Force"
  ],
  "recommended_checks": [
```

```
      "Проверить журнал auth.log на наличие успешных входов с 203.0.113.42",
      "Проверить активные сессии пользователя admin",
      "Проверить историю команд после входа",
      "Проверить наличие 203.0.113.42 в списках известных вредоносных IP",
      "Оценить необходимость блокировки IP-адреса на межсетевом экране"
    ],
    "confidence_note": "Высокая степень уверенности. Паттерн однозначно
  указывает на атаку подбора пароля. Рекомендуется немедленная проверка
  активных сессий.",
    "criticality": {
      "score": 8,
      "level": "high",
      "justification": "Успешная аутентификация после множественных
  неудачных попыток с внешнего IP-адреса указывает на вероятную
  компрометацию учётной записи.",
      "contributing_factors": [
        "Высокое количество неудачных попыток (47)",
        "Успешный вход после серии отказов",
        "Внешний источник атаки",
        "Привилегированная учётная запись (admin)"
      ]
    },
    "response": {
      "action": "contain",
      "urgency": "within_1h",
      "specific_steps": [
        "Завершить все активные сессии пользователя admin с 203.0.113.42",
        "Заблокировать IP-адрес 203.0.113.42 на межсетевом экране",
        "Сбросить пароль учётной записи admin",
        "Проверить наличие изменений в системных файлах",
        "Уведомить команду реагирования"
      ],
      "escalation_needed": true,
      "escalation_reason": "Вероятная компрометация привилегированной
  учётной записи требует командного реагирования"
    }
  }
```


ПРИЛОЖЕНИЕ Г. Конфигурация тестового стенда

Таблица Г.1 — Переменные окружения серверной части

Переменная	Описание	Значение по умолчанию
LM_STUDIO_BASE_URL	Адрес API LM Studio	http://localhost:1234/v1
LM_STUDIO_MODEL	Идентификатор загруженной модели	—

Переменная	Описание	Значение по умолчанию
WAZUH_API_URL	Адрес API Wazuh Manager	—
WAZUH_API_USER	Имя пользователя API Wazuh	wazuh-wui
WAZUH_API_PASSWORD	Пароль API Wazuh	—
WAZUH_VERIFY_SSL	Проверка TLS-сертификата Wazuh	false
WAZUH_INDEXER_URL	Адрес Wazuh Indexer	—
WAZUH_INDEXER_USER	Имя пользователя Indexer	admin
WAZUH_INDEXER_PASSWORD	Пароль Indexer	—
DATABASE_URL	Строка подключения к PostgreSQL	postgresql+asyncpg://...
OSQUERY_TRANSPORT	Транспорт osquery: ssh или mock	mock
OSQUERY_SSH_USER	Пользователь SSH для osquery	root
OSQUERY_SSH_KEY_PATH	Путь к ключу SSH	—
OSQUERY_SSH_TIMEOUT	Тайм-аут SSH (секунды)	10
CORRELATION_TIME_WINDOW_MINUTES	Окно темпоральной корреляции	15
LOG_LEVEL	Уровень журналирования	INFO

Таблица Г.2 — Состав контейнеров Docker Compose

Контейнер	Образ	Порты	Назначение
db	postgres:16	127.0.0.1:5432	Реляционная БД
wazuh.indexer	wazuh/wazuh-indexer:4.9.2	9200 (внутренний)	Хранилище оповещений
wazuh.manager	wazuh/wazuh-manager:4.9.2	1514, 1515, 514/UDP, 55000	Обработка и управление
wazuh.dashboard	wazuh/wazuh-dashboard:4.9.2	8443	Веб-интерфейс Wazuh
backend	Dockerfile (Python 3.12)	8000	Серверная часть системы